

# Peripheral Adapter Register

# Peripheral Adapter Register

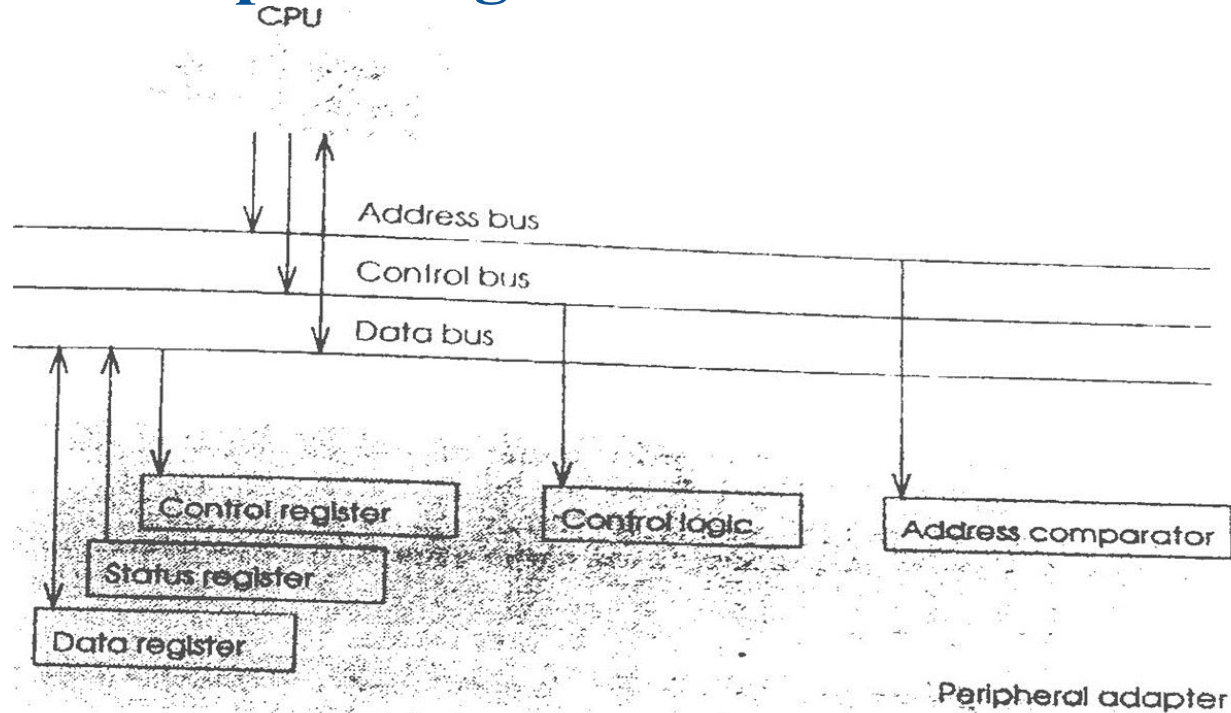


Figure 2.2 Connection of peripheral adapter to highway

# Peripheral Adapter Register

Peripheral adaptors are directly connected to the buses.

PA contains:

- Address Comparator.
- Control register
- Status register
- Data register
- Control Logic

# Peripheral Adapter Register

## ❖ Address Comparator

- It is necessary to distinguish between adapters that are used for different peripherals.
- Address lines are compared with a values allocated to the adaptor
- An adaptor may recognize a single address or a small group of address.
- It compares the address on the address bus with the peripheral adapter's address.

## ❖ Control logic

- If the address on the bus matches the adapter address then the control lines are interpreted by the *control logic* to perform the required function.
- It is connected to a data bus typically to read from or write to a *register* (may be data register)

# Peripheral Adapter Register

## ❖ Control Register

- Stores values written to it to control the operation of the adaptor (Receive, transmit, interrupt, mode).

## ❖ Status Register

- Can be read by the CPU to determine the status of the device (e.g. **whether it is ready for use, busy, error, buffer overflow, transmission complete, receive complete etc.**)
- Each piece of information stored in the registers usually needs only a single bit
- Several such bits are stored in each register, each bit is often known as a *flag*.

## ❖ Data Register

- Used to **hold temporarily a value** to be transferred to or from the peripheral
- So that it is not necessary to synchronize the computer with the peripheral.

# **Computer Input/Output Operation: Programmed I/O**

# Programmed I/O

- The programmed I/O was the simplest type of I/O technique for the exchanges of data or any types of communication **between the processor and the external devices.**
- With programmed I/O, data are exchanged between the processor and the I/O module.
- The instruction set for the CPU typically contains instructions such as
  - **Input data to the CPU from the peripheral**
  - **Output data from the CPU to the peripheral**
  - **Set individual bits in the peripheral adapter's control register**
  - **Test individual bits in the peripheral adapter's status register**

# Programmed I/O : Output

- Output of the data item is straightforward
- The CPU addresses the adapter and tests status register.
- If the relevant bit in the register shows that the device is ready then CPU puts data into the peripherals.
- If the peripheral is not ready, then the program must wait and try again.



# Programmed I/O : Input

- For data input, the status of the input device may be tested and data taken from the input data register if it is available.
- If a number of peripherals are active at the same time, it is necessary to test each one for readiness.
- The technique of testing a number of peripherals in turn is known as *software polling*.

# Programmed I/O : Example

- Processor instructions for communicating with peripherals are often given obvious mnemonics such as “IN” and “OUT”

OUT DATA,AL

(a)

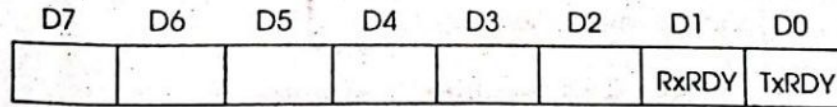
IN AL,STATUS

(b)

**Figure 2.3** 80x86 instructions: (a) output; (b) input

# Programmed I/O : Example

- Processor instructions for communicating with peripherals are often given obvious mnemonics such as “IN” and “OUT”



(a)

```
STATUS EQU OFFH ; Address of status register
DATA EQU OFEH ; Address of data register

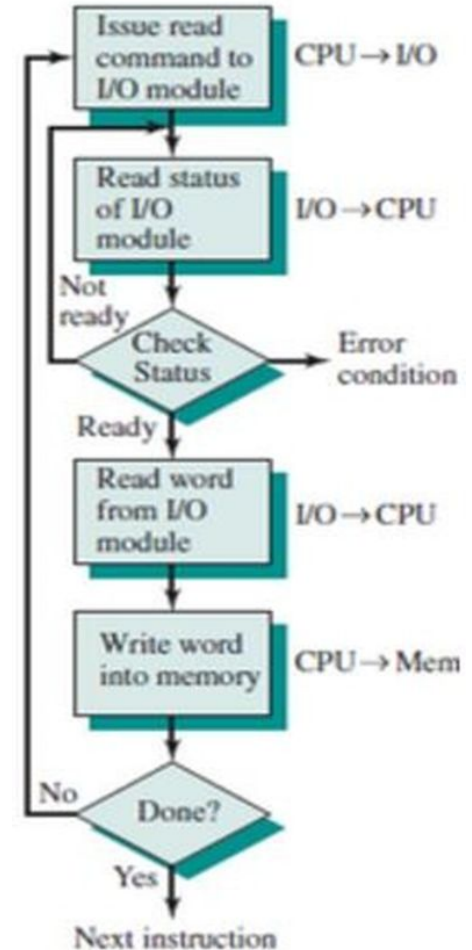
CHECK: IN AL,STATUS ; Get status register
TEST AL,1B ; Test TxRDY bit
JZ CHECK ; Repeat until ready
MOV AL,CHARACTER ; Get the character to send
OUT DATA,AL ; Output to the data register
; for transmission
```

(b)

Figure 2.4 Transmitting a character via an 8251; (a) schematic view of the status register; (b) program used

# Programmed I/O : Data Transfer

- In this case, the I/O device does not have direct access to the memory unit.
- A transfer from I/O device to memory requires the execution of several instructions by the CPU, including an input instruction to transfer the data from device to the CPU and store instruction to transfer the data from CPU to memory.
- In programmed I/O, the CPU stays in the program loop until the I/O unit indicates that it is ready for data transfer.
- This is a time consuming process since it needlessly keeps the CPU busy.



# **Computer Input/Output Operation: Interrupt**

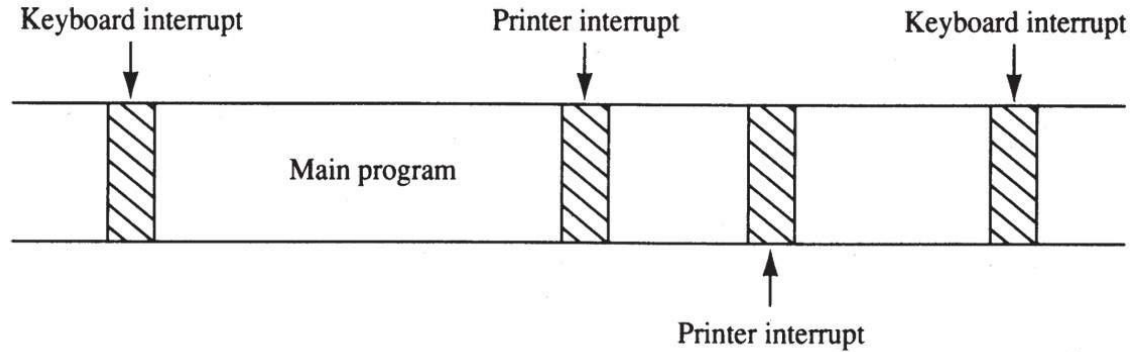
# Interrupt

- ❖ An interrupt is a hardware or software generated change of flow within the system.
- ❖ The main purpose of the interrupt processing are:
  - Interrupts are useful when interfacing I/O devices at **relatively low data transfer rates**, such as keyboard inputs
  - It reduces the waste of time
  - It allows simultaneous execution of softwares

# Interrupt : Example

- ❖ Interrupt processing allows the processor to execute other software while the keyboard operator is **thinking about** what to type next.
- ❖ When a key is pressed, the keyboard encoder debouncing a **switch** and puts out one **pulse** that **interrupts** the microprocessor.

**Figure 12–1** A **time line** that indicates **interrupt usage** in a typical system.

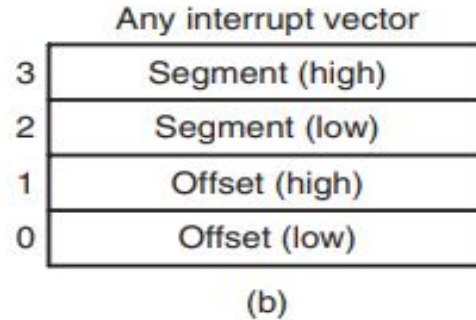


- a **timeline** shows typing on a keyboard, a printer removing data from memory, and a program executing
- the keyboard **interrupt service procedure**, called by the keyboard interrupt, and the **printer interrupt service procedure** is called by Printer interrupt
- each take little time to execute



# Interrupt Vectors

- An **interrupt vector** contains the **address (segment and offset)** of the **interrupt service procedure**.



- The **interrupt vector table** is located in the first 1024 bytes of memory at addresses 000000H–0003FFH.
- contains 256 different four-byte interrupt vectors

# Interrupt Vectors

|      |                                         |
|------|-----------------------------------------|
|      |                                         |
|      | Type 32 — 255<br>User interrupt vectors |
| 080H | Type 14 — 31<br>Reserved                |
|      |                                         |
| 040H | Type 16<br>Coprocessor error            |
| 03CH | Type 15<br>Unassigned                   |
| 038H | Type 14<br>Page fault                   |
| 034H | Type 13<br>General protection           |
| 030H | Type 12<br>Stack segment overrun        |
| 02CH | Type 11<br>Segment not present          |
| 028H | Type 10<br>Invalid task state segment   |
| 024H | Type 9<br>Coprocessor segment overrun   |
| 020H | Type 8<br>Double fault                  |
| 01CH | Type 7<br>Coprocessor not available     |
| 018H | Type 6<br>Undefined opcode              |
| 014H | Type 5<br>BOUND                         |
| 010H | Type 4<br>Overflow (INTO)               |
| 00CH | Type 3<br>1-byte breakpoint             |
| 008H | Type 2<br>NMI pin                       |
| 004H | Type 1<br>Single-step                   |
| 000H | Type 0<br>Divide error                  |

(a)

- Intel reserves the first 32 interrupt vectors
- the last 224 vectors are user-available
- each is four bytes long in real mode and contains the starting address of the interrupt service procedure.

# Intel Dedicated Interrupts

- **Type 0**

The **divide error** whenever the result from a division overflows or an attempt is made to divide by zero.

- **Type 1**

**Single-step** or trap occurs after execution of each instruction if the trap (TF) flag bit is set.

- **Type 2**

The **non-maskable interrupt** occurs when a logic 1 is placed on the NMI input pin to the microprocessor.

- non-maskable—it cannot be disabled

- **Type 3**

A special one-byte instruction (**INT 3**) used to store a **breakpoint** in a program for **debugging**

- **Type 4**

**Overflow** is a special vector used with the INTO instruction. The INTO instruction interrupts the program if an overflow condition exists.

- as **reflected** by the overflow flag (OF)

# Type 5

- The **BOUND** instruction compares a register with **boundaries** stored in the memory.
- If the contents of the register are greater than or equal to the first word in memory and less than or equal to the second word, no interrupt occurs because the contents of the register are within bounds. if the contents of the register are **out of bounds**, a **type 5 interrupt ensues**
- For example, if the instruction BOUND AX,DATA is executed, AX is compared with the contents of DATA and DATA+1 and also with DATA+2 and DATA+3. If AX is less than the contents of DATA and DATA+1, a type 5 interrupt occurs. If AX is greater than the contents of DATA+2 and DATA+3, also type 5 interrupt occurs.

# Interrupt Instructions: **BOUND**, **INTO**, **INT**, **INT 3**, and **IRET**

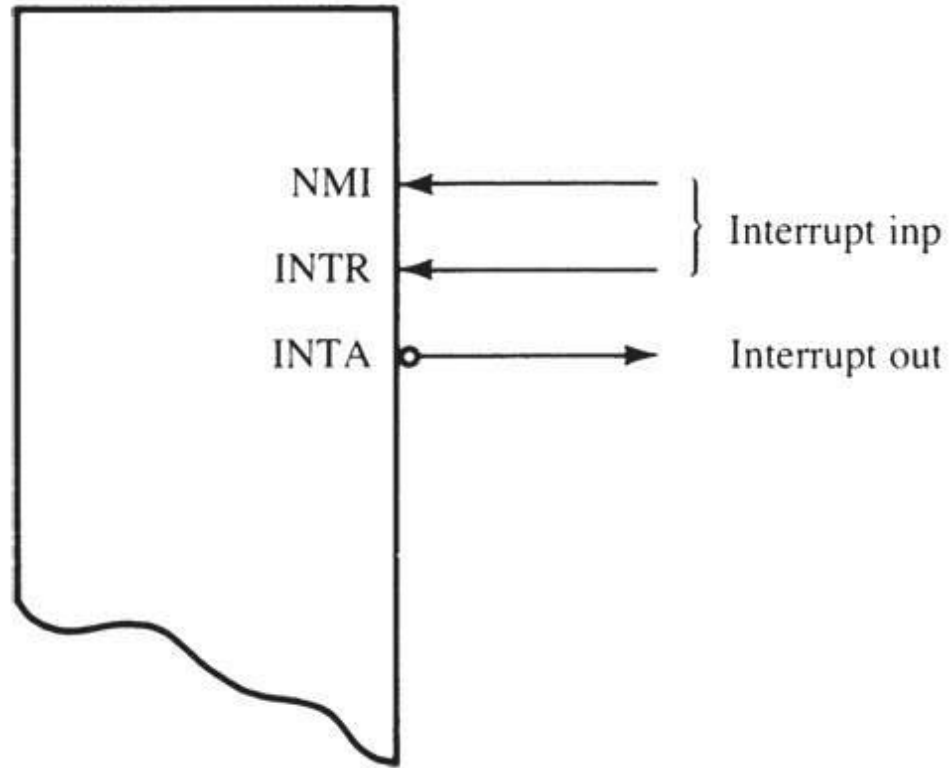
- **BOUND** : Check array against boundary
- **INTO**: The INTO instruction checks or tests the overflow flag (O). If  $O = 1$ , the INTO instruction calls the procedure whose address is stored in interrupt vector type number 4.
- **INT n**: INT n instruction calls the interrupt service procedure that begins at the address represented in vector number n. For example,  $INT\ 5 = 4 \times 5$  or 20 (14H). The vector for INT 5 begins at address 0014H and continues to 0017H
- **INT 3: Interrupt 3**
- **IRET**: used to return for both software and hardware interrupts.

# 12–2 HARDWARE INTERRUPTS

- The two processor hardware interrupt inputs:
  - non-maskable interrupt (NMI)
  - interrupt request (INTR)
- One interrupt output:
  - INTA



**Figure 12–5** The interrupt pins on all versions of the Intel microprocessor.



## The **non-maskable interrupt** (NMI):

- It is an input pin that requests an interrupt.
- The NMI pin must remain logic 1 until recognized by the microprocessor.
- This type of interrupts can not be avoided.
- The NMI input is often used for parity errors and other major faults, such as power failures.

- **Interrupt request input (INTR):** It is used for requesting an interrupt. It must be held at a logic 1 level until it is recognized. INTR is set by an external event and cleared inside the interrupt service procedure. INTR is automatically disabled once accepted and re-enabled by IRET at the end of the interrupt service procedure.
- **INTA:** The processor responds to INTR by pulsing INTA output. It is used to apply interrupt vector type number on data bus connections  $D_7-D_0$ .

# Operation of a Real Mode Interrupt

- When the processor **completes executing the current instruction**, it determines whether an interrupt is active **by checking**:
  - (1) instruction executions
  - (2) single-step
  - (3) NMI
  - (4) coprocessor segment overrun
  - (5) INTR
  - (6) INT instructions in the order presented

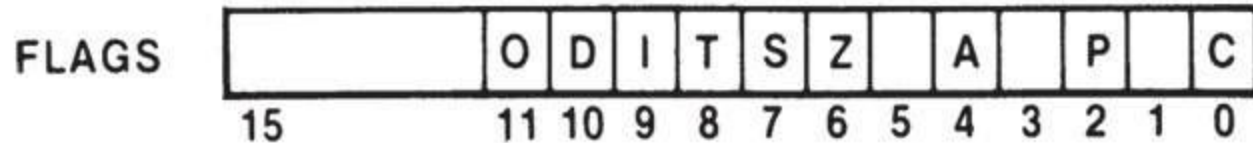
- **If one or more are present:**

- 1. Flag register contents are **pushed** on the stack
- 2. Interrupt (IF) & trap (TF) flags **clear, disabling the INTR pin** and trap or single-step feature
- 3. Contents of the code segment register (CS) are **pushed** onto the **stack**
- 4. Contents of the instruction pointer (IP) are **pushed** onto the **stack**
- 5. **Interrupt vector** contents are fetched and placed into **IP** and **CS** so the next instruction executes at the interrupt service procedure addressed by the vector

# Interrupt Flag Bits

- IF: when IF is **set**, it *allows* the INTR pin to cause an interrupt. When IF is **cleared**, it *prevents* the INTR pin from causing an interrupt
- 
- TF: when  $TF = 1$ , it causes a trap interrupt (type 1) to occur after each instruction executes. Trap is often called a *single-step*. when  $TF = 0$ , normal program execution occurs

- the interrupt flag is set and cleared by the STI and CLI instructions, respectively
- the contents of the flag register and the location of IF and TF are shown here



**Figure 12–4** The flag register. (Courtesy of Intel Corporation.)

# Priority Interrupt

- Whenever a peripheral is ready to transfer data , it is necessary to service its interrupt within a reasonable time.
  - Otherwise subsequent data may be lost
- **Faster peripheral require faster servicing**
- If two or more peripherals are ready at the same time, it is better to **service the faster one first** since the slow one can be made to wait a little while
- Multiple interrupt requests can be resolved by using a priority interrupt scheme
- A **hardware priority encoder** arbitrates between requests and sends a single value- that **represents the highest priority device-** to the CPU
- The interrupt request is formed by performing a **logical OR** of the peripherals IREQ lines.



# Priority Interrupt

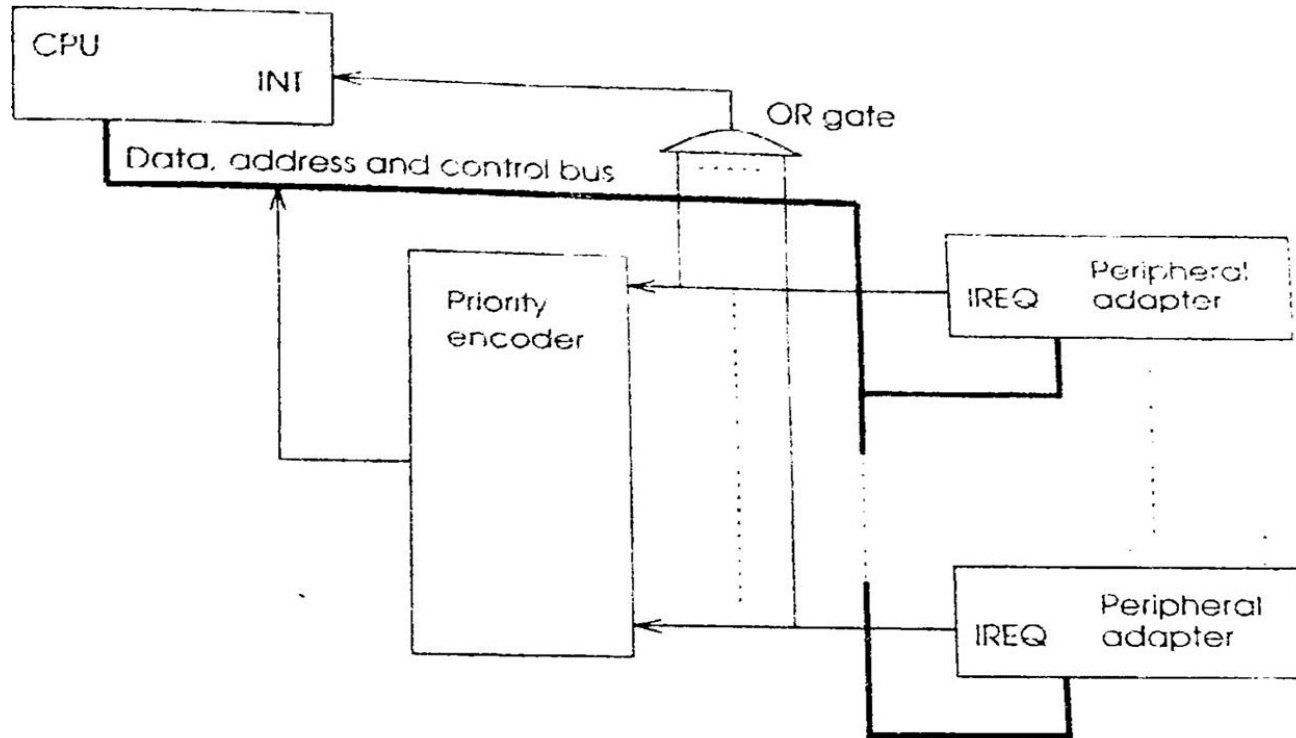


Figure 2.6 Priority interrupts using a priority encoder

# Priority Interrupt Problem & Solution

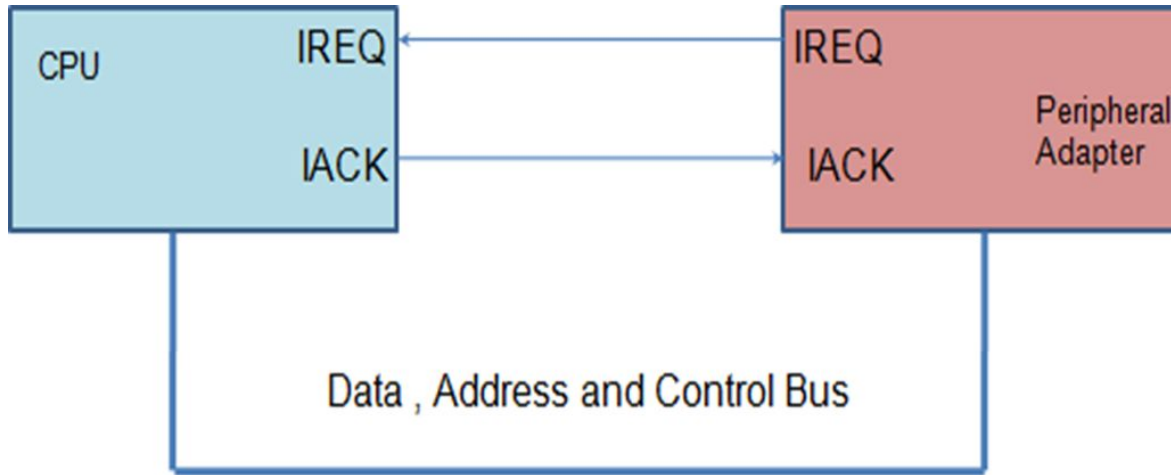
## Problem

- Interrupt requests are assumed to **remain asserted until reset by instructions** in the service routine.
- It is not possible for another request to be seen until a request is de-asserted
- Data from a fast peripheral being lost while a service routine is getting around to clearing a low priority interrupt

## Solution

- Most of the computers have a signal generated by the CPU that is returned to the peripheral as soon as the interrupt is detected- ***Interrupt acknowledge signal (IACK)***.
- This clears the interrupt request from that device and allows other devices to use the interrupt line.

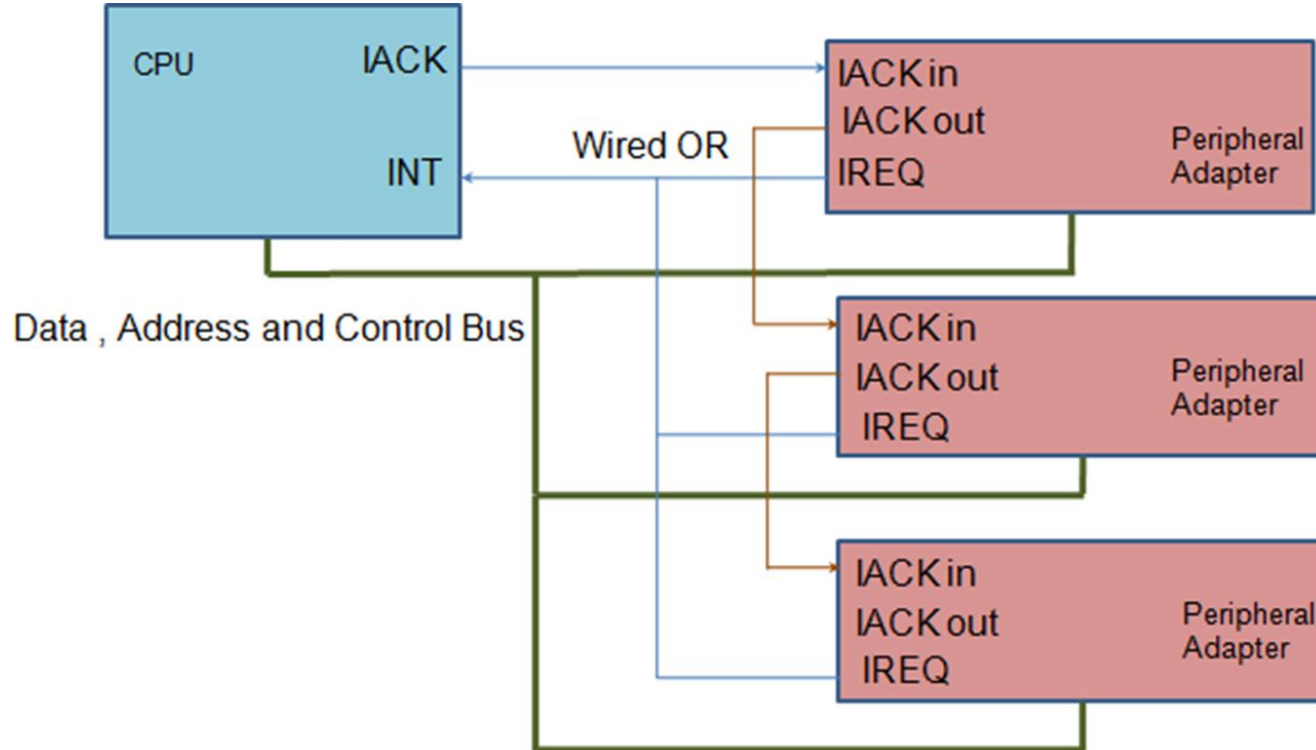
# Interrupt with Acknowledgement



# Priority Interrupt Using Daisy Chain

- Using an interrupt acknowledge enables the construction of priority interrupt scheme.
- CPU is able to **determine priority** not from the interrupt request but **by which device the acknowledge is sent to**.
- All the interrupt request lines are OR together.
- The **CPU IACK is connected directly to the highest priority device** so that if more than one request has been made the highest priority device sees it first.
- If it has not made a request, **it passes the IACK along to the next device**
- This continues down to the lowest priority device.
  - This receives an acknowledgement only if no other device has made a request.

# Priority Interrupt Using Daisy Chain

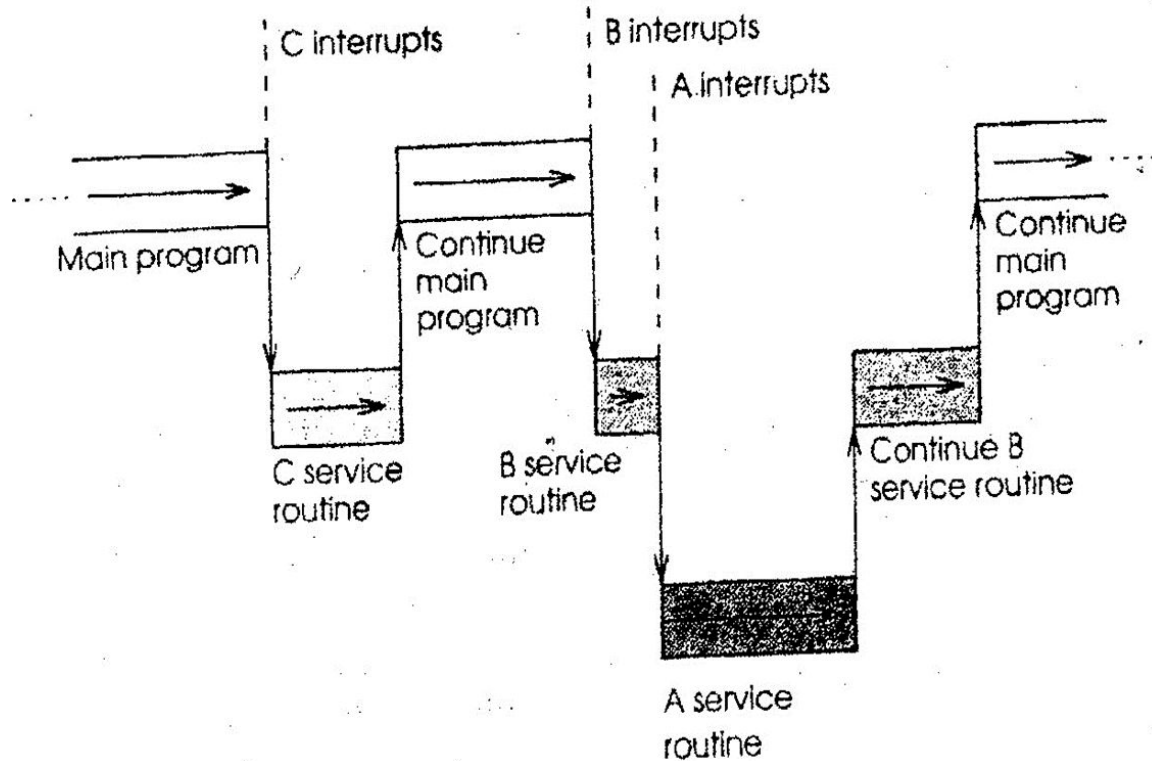


# Nested Interrupt

- In order to ensure a fast response to a high priority interrupt it is possible to allow interrupts to interrupt interrupts.

.

# Nested Interrupt



# Nested Interrupt

- Device C interrupt the main program, it's service procedure runs to the completion, returning to the main program.
- Device B similarly interrupt the main program but is,. In turn, interrupted by a higher priority device, A.
- Execution of the service procedure for A is nested within that for B
- When the service routine for device A complete the CPU returns to perform the service routine for B
- When it completes the main program continues.

.



**Thank You**



**Military Institute of Science and Technology**  
**Department of Computer Science and Engineering**

# **Peripheral/Computer Connections**

## **(Part 2)**

---

**Course Name: Computer Interfacing**  
**Course Code: CSE-405**

# Book Reference

- ❏ Computer Peripherals (3rd Edition) - Cook and White; Butterworth- Heinemann (1995)

# Block Data Transfer

- Processors have instructions for moving **blocks of data** which may be used for I/O.
- **CPU has to synchronize** its block moves with the peripheral and is unable to perform any other functions while the block move is in progress.
- **Only suited for fast transfers** where the CPU and the peripheral speeds are reasonably well matched.
- It is not commonly used.

# Direct Memory Access (DMA)

- It is better to avoid using the CPU completely for I/O transfer.
- A special hardware called **DMA controller** can be used to transfer data between memory and I/O devices.
- When a peripheral indicates that it is ready for data transfer
  - The **DMA unit gains the control of the Bus**.
  - Places **appropriate address and control signals** on it to make the transfer.
  - **Releases the bus**.
- This action of taking over the bus for a period and **executing a memory access cycle** instead of the CPU doing so is known as *cycle stealing*.

# Parallel Interface: SCSI

# Parallel Interface: SCSI

- **SCSI (Small Computer System Interface) is a set of standards for physically connecting and transferring data between computers and peripheral devices especially storage devices.**
- Used to connect computers to disk drives and other peripherals (generally those needing a high data rate).
- A complete communication protocol and bus standard
- **8 devices can be supported on the SCSI bus**
  - Each device can have 8 logical units
  - Each logical unit contains a unique Logical Unit Number (LUN)
  - Each logical unit can have 256 logical sub units
    - Thus a very large total number of possible devices

# Parallel Interface: SCSI

- Each device can be either an **initiator**, a **target** or both.

## Initiator:

- A SCSI device that requests an operation to be performed **by another SCSI device**.
- Initiate a SCSI session.
- Does not provide any Logical Unit Number.

## Target:

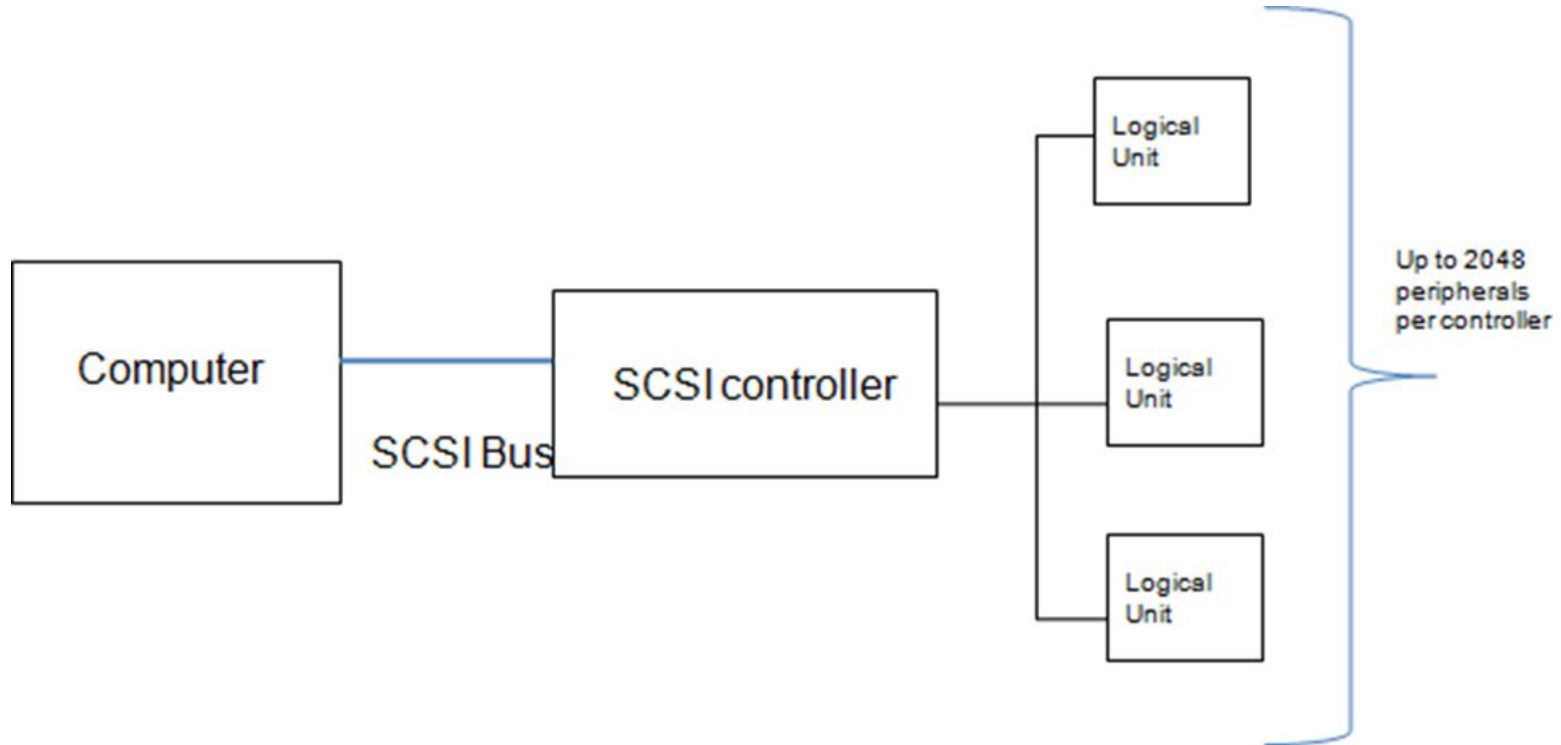
- A SCSI device that performs an operation as requested by an initiator.
- Waits for initiators' commands and provides required input/output data transfers.
- Provide one or more Logical Unit Number.



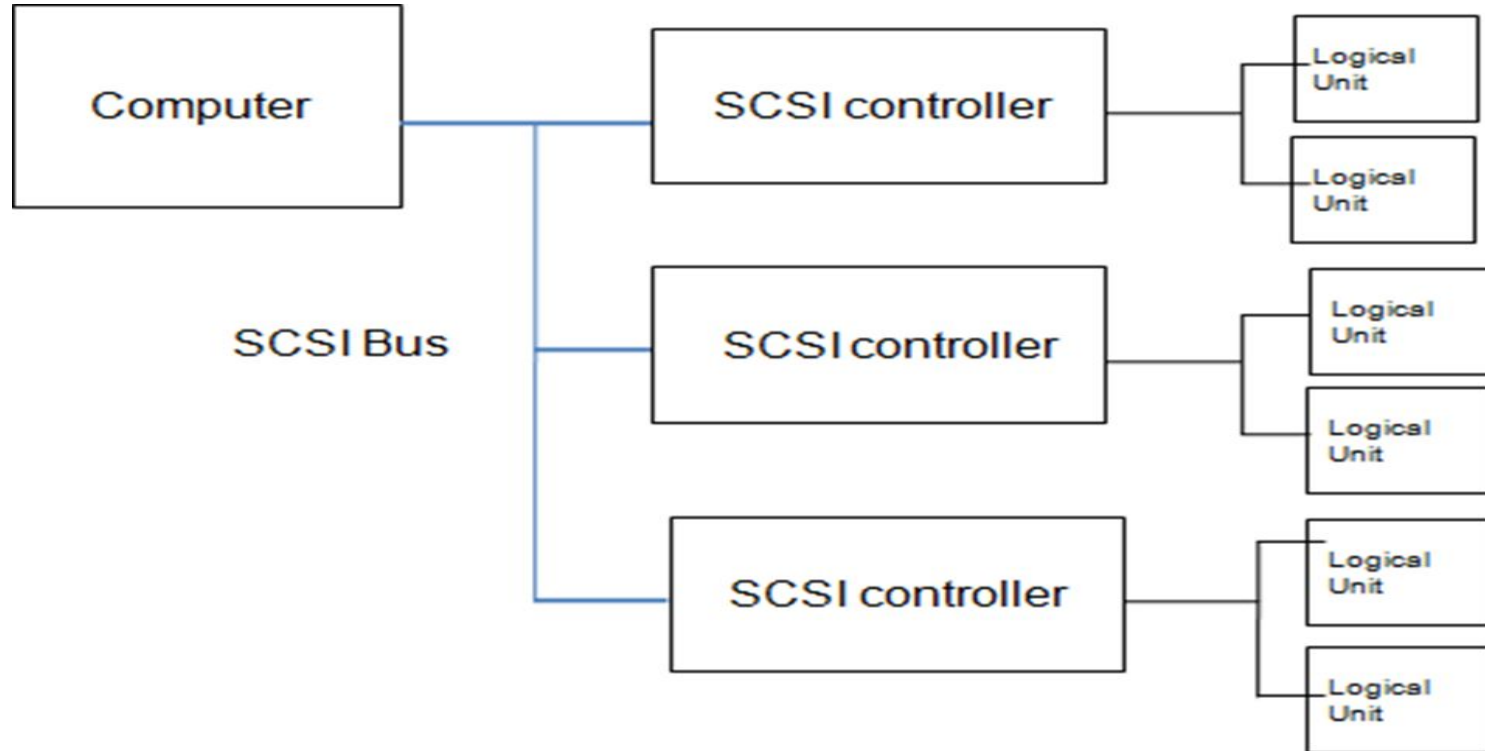
# Parallel Interface: SCSI

- To be useful SCSI bus **must have at least one initiator and one target**
  - Can have multiple initiator and/or target
- Data is transferred over an **8 bit bidirectional bus**
  - **Optional parity line to check the error** of transmission
- **Data and control lines are implemented using one wire each together with common ground in single-ended SCSI**
  - With pair of wires for each line in differential SCSI
- When an **end user sends a request**, the **operating system sends the SCSI commands to the SCSI controller**, which then sends it to the storage device.
- **SCSI controller** (embedded on motherboard) is a **host bus adapter** or a chip that allows a SCSI storage device to communicate with operating system across a SCSI bus.

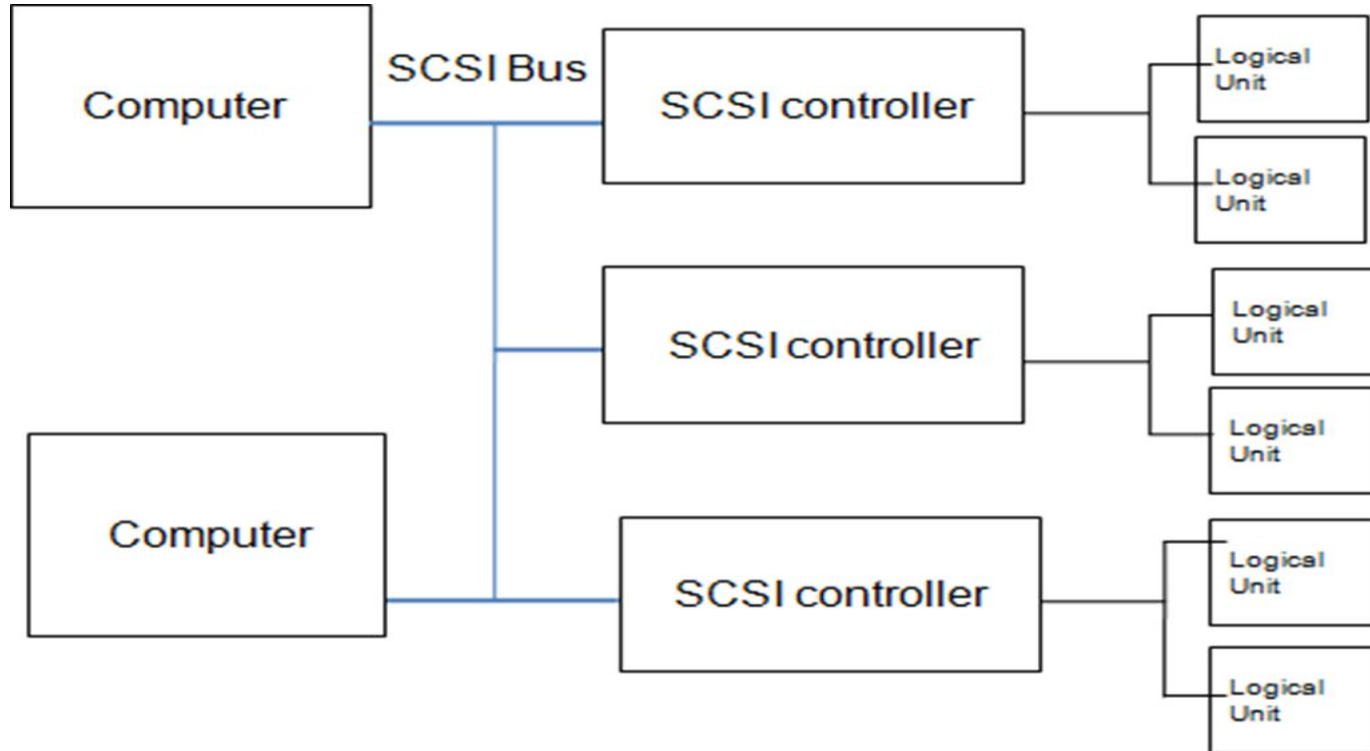
# One Initiator, One Target



# One Initiator, Multiple Targets



# Multiple Initiators, Multiple Targets



# Protocol for Using SCSI Bus

- Phase 1 : Arbitration
  - Phase 2 : Selection
  - Phase 3 : Information Transfer
  - Phase 4 : Bus free
  - Phase 5 : Reselection
- 
- Several commands can be received before the first is completed
  - Devices can queue requests

# Protocol for Using SCSI Bus

## Phase 1: Arbitration

- Allows one SCSI device (here initiator) to gain control of the SCSI bus so that it can initiate or resume an I/O process.
- The **SCSI device shall first wait for the BUS FREE phase** to occur

## Phase 2: Selection

- The SELECTION phase **allows an initiator to select a target** for the purpose of initiating some target function.

## Phase 3: Information transfer

- The COMMAND, DATA, STATUS, and MESSAGE phases are combined to transfer data or control information.

# Protocol for Using SCSI Bus

## Phase 4: Bus Free

- BUS FREE phase is entered when a **target release the BUSY signal**
- The BUS FREE phase indicates that **there is no current I/O process and the SCSI bus is available for a new connection.**

## Phase 5: Reselection

- When the target is ready to respond to the given command then it can re-establish contact with the initiator in Reselection phase.
- Connection that started by the initiator but was **suspended by the target.**

# SCSI: Mathematical Problem-1

| Initiator | Target | Arbitration time of initiator (sec) | Task time of target given by initiator (sec) |
|-----------|--------|-------------------------------------|----------------------------------------------|
| 2         | 1      | 4                                   | 8.4                                          |
| 1         | 3      | 5.1                                 | 5                                            |
| 1         | 2      | 6.4                                 | 5.8                                          |
| 1         | 3      | 8                                   | 3.5                                          |
| 2         | 2      | 8.2                                 | 4.9                                          |

Suppose, 2 computers are sharing a common SCSI bus among them. 3 SCSI controllers are connected with the same bus. Now solve the following scenario and find out the value of  $t$  at which all the controllers and bus will become free after executing all the tasks given by the all the computers.

**Consider ,data transfer rate between all components = 1.7 sec**

**And arbitration +selection/reselection time= 2 sec**



## SCSI: Mathematical Problem-2

| Initiator | Target | Arbitration time of initiator (sec) | Task time of target given by initiator (sec) |
|-----------|--------|-------------------------------------|----------------------------------------------|
| 2         | 3      | 1                                   | 10                                           |
| 1         | 2      | 3                                   | 8                                            |
| 1         | 3      | 6                                   | 4                                            |
| 3         | 1      | 8                                   | 20                                           |
| 3         | 3      | 12                                  | 9                                            |

Suppose, 3 computers are sharing a common SCSI bus among them. 3 SCSI controllers are connected with the same bus. Now solve the following scenario and find out the value of  $t$  at which all the controllers and bus will become free after executing all the tasks given by the all the computers.

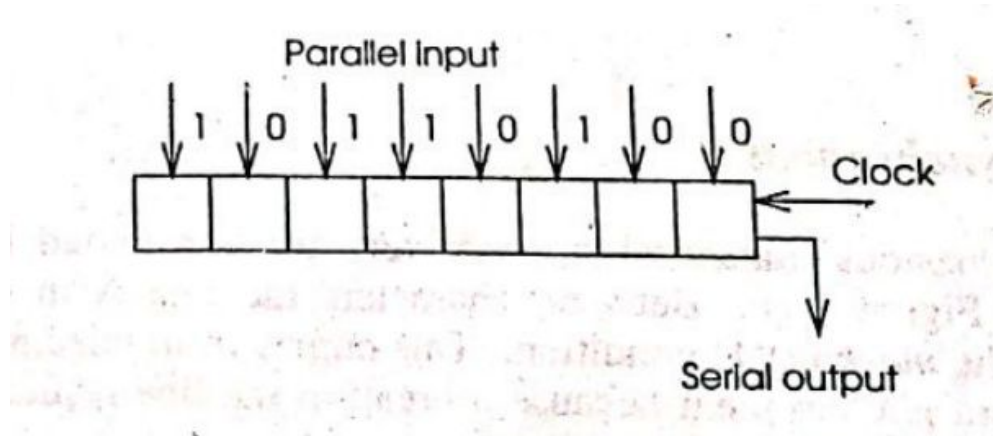
**Consider ,data transfer rate between all components = 2 sec**

**And arbitration +selection/reselection time= 1 sec**

# Serial Interface

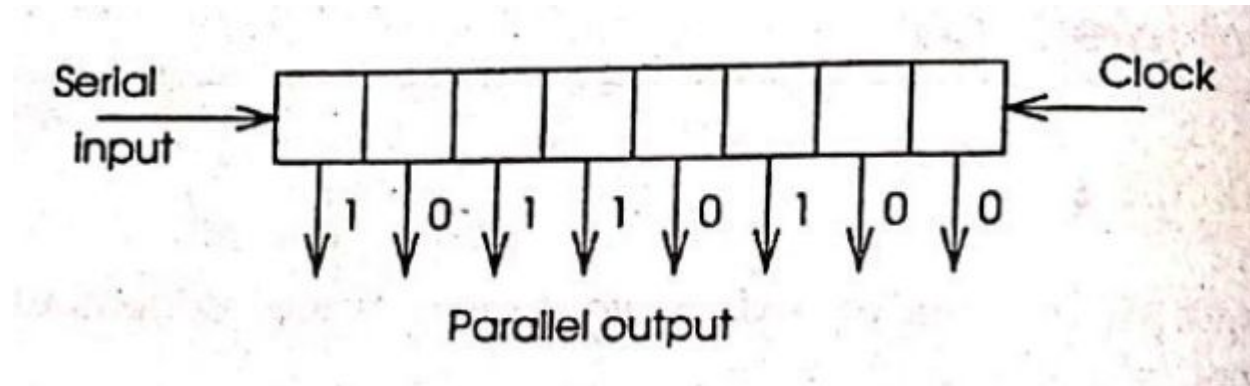
# Parallel to Serial Interface

- Simplest convertor is a **shift register**
- Parallel data word is loaded to into a shift register
- A pulse on the clock input shift the data by 1 bit. And output 1 bit.

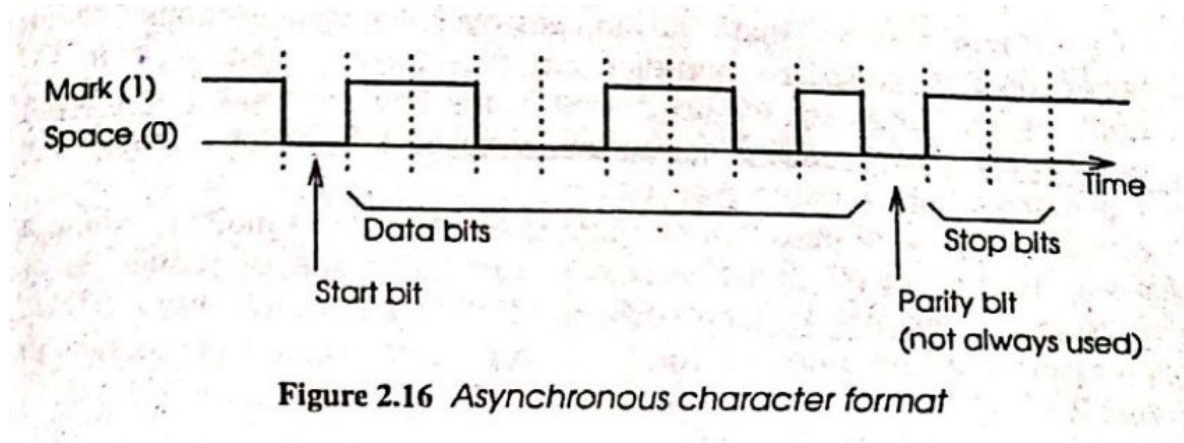


# Serial to Parallel Interface

- Sequence of n clock pulse caused the input to propagate along the shift register
  - Until it is all available in parallel
  - First bit shifted all the way through shift register and appears at the right end.
- **The clock in the receiver must operate with the same timing as the transmitter.**



# Asynchronous Transmission



# Asynchronous Transmission

- Between characters the lines are in the **idle state** which is the '**mark**' or '**1**' condition
- The first bit of the character, the '**start bit**' is in the '**space**' or '**0**' condition
- The 1 to 0 transition is used to start the receiver clock and indicates that a character is arriving
- The **first bit of data can not be the start bit**
- Because if the **first bit is 1** it will be **indistinguishable from the idle condition**
- For the data part of the character 5,6,7 or 8 data bits are transmitted with least significant bit first
- There may be a **parity bit to allow transmission error** to be detected
- Finally, the line returns to a idle state for **1, or 2 stop bits which allow the receiver to settle** ready for the next character
- Also known as 'stop-start' transmission

# Asynchronous Transmission

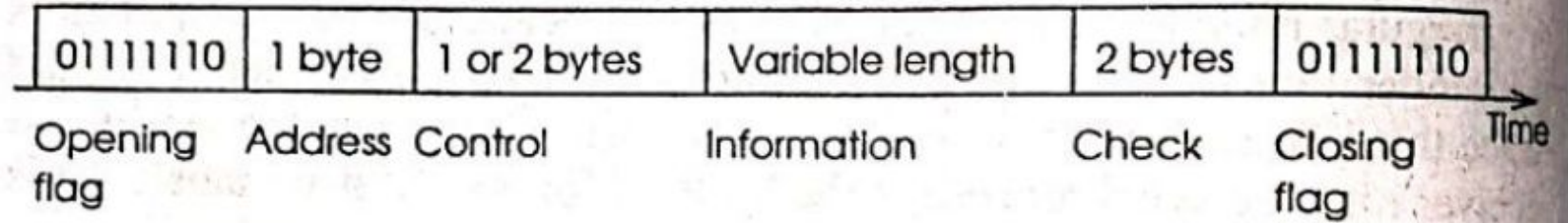
- Irregular intervals may occur between character
- **The sender and the receiver are not synchronized at the character level**
- Hence known as asynchronous transmission

# Synchronous Transmission

- Supply a **clock signal** and a **data signal**
- Transmission may be along a **pair of line, one for clock signal and one for data**
- Clock may be merged with data stream into a single line
- Two advantages:
  1. **No time wasted** for non-data bits
  2. **Less margin of error** is required to ensure correct recovery of data- increase acceptable **transmission speed**
- **Use a special sequence of data known as protocols**
- One protocol: HDLC (High Level Data Link Control)

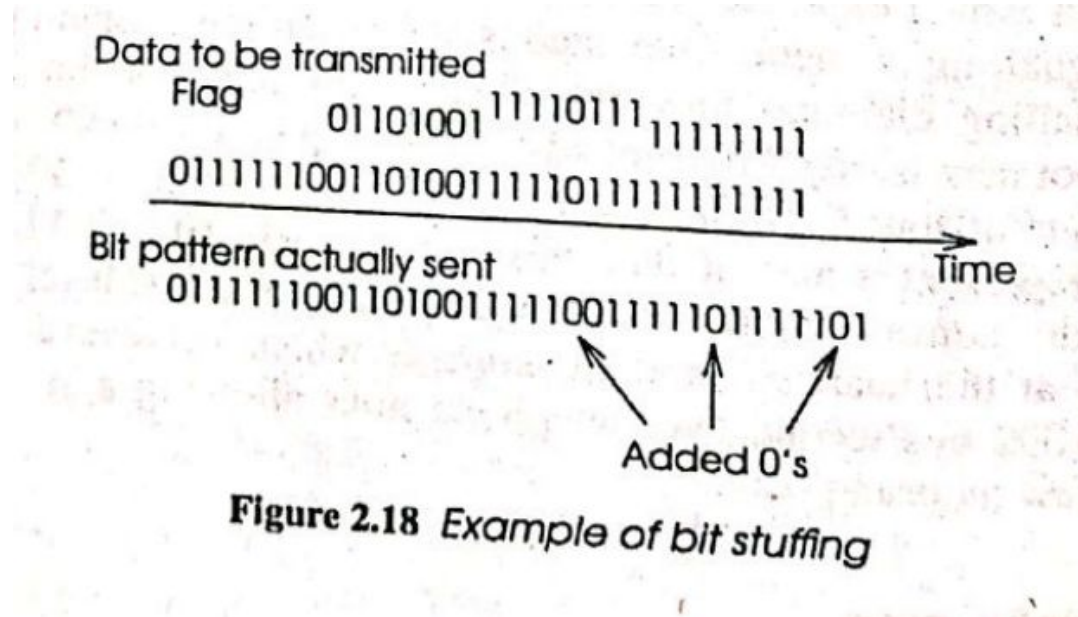


# Synchronous Transmission : HDLC format



**Figure 2.17** HDLC synchronous format

# Synchronous Transmission: Bit stuffing



**Figure 2.18** Example of bit stuffing